Code Review Process

✅ Standard Code Review Process (for peers):

1. **Understand the Purpose**
   - Read the PR/commit message
   - Understand the feature, bug fix, or refactor scope
2. **Checklist for Review:**

🔍 Code Quality

- Clear, readable, and maintainable
- Follows naming conventions
- No commented-out code or unused imports

🧠 Logic & Functionality

- Business logic is correct
- Handles edge cases and errors gracefully
- No redundant or duplicated logic

⚙️ Performance

- Efficient data structures and algorithms
- Avoids unnecessary processing or memory usage

🛡️ Security

- Input validation and sanitation
- No hardcoded secrets or sensitive info
- Proper authorization checks

🧪 Testing

- Unit/integration tests included and pass
- Tests cover important paths and edge cases

📦 Design & Structure

- Follows architecture and design patterns
- Good modularity, separation of concerns

📄 Standards & Conventions

- Follows project's coding standards
- Proper formatting and consistent style

🔁 Final Steps:

- Add comments or suggestions clearly
- Ask questions if logic is unclear
- Approve, request changes, or escalate if needed

Akash Kumar

✅ Code Review Checklist

1. Functionality

- Does the code meet the requirements/specifications?
- Does it handle all expected inputs and edge cases?
- Are there any logical bugs or missing conditions?

2. Code Quality & Readability

- Is the code clean and readable?
- Are variable/method/class names meaningful and consistent?
- Is the code modular and broken into small, reusable functions?

3. Error Handling

- Are errors handled gracefully (try-catch, fallbacks)?
- Are informative messages/logs used where needed?

4. Performance

- Are data structures and algorithms optimal?
- Are there any obvious memory or CPU bottlenecks?

5. Security

- Is user input properly validated/sanitized?
- Are there any hardcoded secrets or credentials?
- Is access control and authentication handled properly?

6. Testing

- Are unit and/or integration tests included?
- Do the tests cover positive, negative, and edge cases?
- Do all tests pass?

7. Best Practices & Standards

- Is the code aligned with project/organization standards?
- Are there any anti-patterns used (e.g., deeply nested loops, magic numbers)?
- Is exception handling done properly (not silently swallowing)?

8. Documentation & Comments

- Are functions/methods properly documented?
- Are comments used where logic is complex?
- Is there a README/update if necessary?

9. Dependencies

- Are new dependencies justified and reviewed?
- Are outdated or unnecessary dependencies removed?

10. Version Control

- Is the commit history clean and meaningful?
- Are there no unnecessary debug logs or commented-out code?

Akash Kumar

🔐 Security Checklist During Code Review

1. Input Validation

- Are all user inputs validated on both frontend and backend?
- Are inputs sanitized to prevent **SQL Injection**, **XSS**, etc.?

2. Authentication & Authorization

- Is authentication properly enforced for sensitive endpoints?
- Are authorization checks in place to prevent **privilege escalation**?

3. Secrets Management

- No hardcoded API keys, passwords, or secrets in the code
- Are secrets stored in secure config (e.g., env vars, Vault)?

4. Data Protection

- Is sensitive data (PII, tokens, etc.) encrypted at rest and in transit?
- Are logs free from sensitive data?

5. Secure Dependencies

- Are dependencies up to date (check with tools like OWASP Dependency-Check, npm audit, or mvn dependency:check)?
- No use of vulnerable libraries?

6. Error Handling

- Are exceptions handled gracefully without leaking sensitive info?
- Is the system resilient to invalid or malicious inputs?

7. HTTP Security

- Are security headers (e.g., Content-Security-Policy, X-Frame-Options) in place?
- Is HTTPS enforced?

🛠️ Automated Tools (Recommended)

Use these tools for automated security scans:

- **OWASP Dependency-Check** – library vulnerabilities
- **SonarQube** / **Checkmarx** – code security scanning
- **Snyk** – dependency scanning
- **FindSecBugs (Java)** – static analysis for Java security issues
- **Bandit (Python)** – Python code security scanner

✅What is the DRY Principle?

DRY stands for **Don't Repeat Yourself**.
It's a software development principle aimed at reducing code duplication by abstracting and reusing logic.

📌 Usage

- Reuse common logic in **functions/methods**, **classes**, or **utilities**
- Apply **inheritance**, **composition**, or **helper services**
- Extract repetitive **SQL**, **configurations**, or **HTML** into templates/components

Akash Kumar

💡 Benefits

✅ Easier **maintenance** – change in one place updates everywhere
✅ Reduces **bugs** – fewer places to make mistakes
✅ Improves **readability** and **clean architecture**
✅ Encourages **reusability** and **modular design**

❌ Before DRY (Code Duplication Impact)

- 🔁 Multiple copies of the same logic
- ❌ Error-prone – bugs fixed in one place but missed in others
- 🛠️ High maintenance effort
- 😵 Confusion during debugging and onboarding

✍️ Example (Java)

❌ Before DRY:

```
System.out.println("Connecting to database...");
System.out.println("Connecting to database...");
```

✅ After DRY:

```
public void connectToDatabase() {
    System.out.println("Connecting to database...");
}
```

✅What is the KISS Principle and how can we apply for that?

KISS stands for **Keep It Simple, Stupid**.
It's a design principle that emphasizes simplicity in software development, encouraging developers to avoid unnecessary complexity.

Core idea: Simple solutions are easier to understand, maintain, test, and extend.

📌 How to Apply KISS in Code

1. ✅ **Write clear and concise methods**
   - Prefer small, single-responsibility functions
2. ✅ **Avoid over-engineering**
   - Use design patterns only when needed
3. ✅ **Use standard libraries/APIs**
   - Don't reinvent the wheel
4. ✅ **Limit nested logic**
   - Avoid deep `if-else`, loops, and try-catch blocks
5. ✅ **Keep configuration simple**
   - Use default values and meaningful naming

Akash Kumar

💡 Usage

- In **API design** – clean endpoints with limited parameters
- In **frontend/backend logic** – readable flow with no clever hacks
- In **architecture** – prefer monolith until complexity requires microservices

🎯 Benefits of KISS

- ✅ Faster development and debugging
- ✅ Easier onboarding for new developers
- ✅ Lower maintenance cost
- ✅ Reduced chances of bugs due to unnecessary logic

❌ Impact Before Applying KISS (Over-Complex Design)

- 😵 Hard-to-understand logic with over-abstraction
- 🔁 Repetitive code due to fear of changing complex flow
- 🐞 More bugs from convoluted conditions and dependencies
- ⏳ Time wasted understanding the code instead of solving problems

✍️ Example (Java)

❌ Before KISS (Too Complex):

```java
public int calculate(int a, int b, String type) {
    if ("ADD".equals(type)) {
        return a + b;
    } else if ("SUB".equals(type)) {
        return a - b;
    } else if ("MUL".equals(type)) {
        return a * b;
    } else if ("DIV".equals(type)) {
        return a / b;
    } else {
        return 0;
    }
}
```

✅ After KISS (Simple & Extensible):

```
public int calculate(int a, int b, Operation op) {
    return op.apply(a, b);
}
```

✅What is the YAGNI Principle? How can we use it ?

YAGNI stands for You Aren't Gonna Need It.
It's a software development principle that encourages developers to **only implement functionality when it's needed**, not based on future assumptions.
Core Idea: Don't write code for features you *think* you'll need later — build only what is required now.

📌 **How to Apply YAGNI**

1. ✅ Focus only on current requirements
2. ✅ Avoid writing speculative code or premature extensions
3. ✅ Don't over-engineer for future scenarios
4. ✅ Use refactoring when a real need arises
5. ✅ Keep the design flexible, not bloated

💡 **Usage**

- In **feature planning** – build minimal viable features
- In **API design** – avoid adding unused endpoints or parameters
- In **class design** – don't add properties or methods "just in case"

🎯 **Benefits of YAGNI**

- ✅ Reduces **wasted effort** and **development time**
- ✅ Keeps codebase **lean and clean**
- ✅ Increases **agility** – easier to refactor when truly needed
- ✅ Prevents introducing **unused bugs or complexity**

❌ **Impact Before YAGNI (Overplanning)**

- 🛠️ Unused code that still needs to be maintained
- 🐞 Bugs in unused features that add technical debt
- ⏳ Slower development and bloated codebase
- 🤯 Confusion for other developers due to unnecessary complexity

✍️ **Example (Java)**

❌ **Before YAGNI:**

```
// Future-proofed class with unused method
public class PaymentService {
    public void processPayment() { /* ... */ }
    public void refundPayment() { /* unused for now */ }
    public void schedulePayment() { /* unused for now */ }
}
```

Akash Kumar

✅ **After YAGNI:**

```
public class PaymentService {
    public void processPayment() { /* ... */ }
}
```

Standard parameters and principles

✅ 1. Code Readability

- Follow **naming conventions** (camelCase for variables, PascalCase for classes).
- Use **meaningful, descriptive names** for variables, methods, and classes.
- Avoid deep nesting and complex logic in a single block.
- Add **comments only where necessary** to explain "why", not "what".

✅ 2. Clean Code Principles

- Follow **SOLID principles** (especially Single Responsibility Principle).
- Avoid **duplicate code** (DRY principle).
- Use **small, focused functions** (max 10–15 lines).
- Apply the **KISS** principle – Keep It Simple, Stupid.
- Follow the **YAGNI** principle – You Aren't Gonna Need It.

✅ 3. Consistent Style

- Use **formatters and linters** (Prettier, Checkstyle, ESLint, Spotless for Java).
- Maintain consistent **indentation**, **spacing**, and **braces**.
- Adhere to your team or language's **style guide** (e.g., Java Code Conventions, Google Java Style Guide).

✅ 4. Error Handling & Logging

- Handle **exceptions gracefully** using try-catch, and log meaningful messages.
- Avoid swallowing exceptions without logging.
- Do not overuse generic exceptions (Exception, Throwable).
- Use structured logging (log.error("User not found: {}", userId)).

✅ 5. Code Reusability & Modularity

- Avoid hardcoding values — use constants/config.
- Break logic into **reusable, testable methods or services**.
- Design using **interfaces and abstraction** where appropriate.

✅ 6. Test Coverage

- Write **unit tests** for business logic.
- Use **integration tests** for system behavior.
- Follow **AAA structure** (Arrange, Act, Assert).
- Ensure coverage of edge cases and negative scenarios.

Akash Kumar

✅ 7. Performance & Optimization

- Use appropriate **data structures** (e.g., Map vs List).
- Avoid **unnecessary object creation** or **heavy computations in loops**.
- Be aware of **database efficiency** (e.g., N+1 query issues).

✅ 8. Security

- Sanitize inputs to avoid **SQL injection**, **XSS**, etc.
- Avoid exposing sensitive data in logs or APIs.
- Use proper **authentication and authorization** checks.

✅ 9. Documentation

- JavaDocs or KDocs for public APIs.
- Comments only for **non-obvious logic**.
- Update README and configuration guides if new features are added.

✅ 10. Version Control Hygiene

- Clear, concise **commit messages**: Fix NPE when user is null, Add validation for email format.
- Group related changes together.
- Do not commit debug logs, secrets, or commented-out code.

✨ BONUS – Code Review Checklist Summary:

Category	Checklist
Readability	Naming, structure, comments
Functionality	Meets requirements, edge cases handled
Code Quality	Clean, DRY, SOLID, modular
Tests	Unit and integration tests present
Security	Input validation, no secrets in code
Performance	Optimized loops, memory, DB queries
Style	Matches code style guide

Some Other ways

1. **Use Meaningful Variable and Function Names**
   - Avoid single-character names like `x` or `y` (unless in loops).
   - Use names that reflect the purpose, e.g., `totalSales` instead of `ts`.
2. **Follow Consistent Code Formatting**
   - Use proper indentation and spacing.
   - Stick to the style guide of your language (e.g., PEP8 for Python).

Akash Kumar

3. Avoid Hardcoding Values
   - Instead of embedding constants in your code, use named variables or configuration files.
   - Example: Replace `if (score > 90)` with `if (score > HIGH_SCORE_THRESHOLD)`.
4. Keep Functions Short and Focused
   - Each function should perform one task.
   - A good rule of thumb is to limit functions to 20-30 lines of code.
5. Write Comments Wisely
   - Comment only where necessary (don't state the obvious).
   - Explain why something is done, not what it does.
6. DRY Principle (Don't Repeat Yourself)
   - Avoid duplicating code. Extract reusable logic into separate functions or classes.
   - Repeated code is harder to maintain and prone to bugs.
7. Optimize for Readability Over Cleverness
   - Clear code is better than fancy but confusing code.
   - Write in a way that even junior developers can understand.
8. Handle Errors Gracefully
   - Use proper error handling to avoid application crashes.
   - Example: Validate inputs and handle edge cases effectively.
9. Test Your Code Regularly
   - Write unit tests to ensure your code works as expected.
   - Test edge cases and unusual inputs to improve reliability.
10. Refactor When Necessary
    - Regularly review and improve your codebase.
    - Clean up old code to make it more efficient and maintainable.

Additional Tips:

- Use tools like linters (e.g., ESLint, Pylint) to catch formatting errors.
- Collaborate with peers to get feedback on your coding style.

Basic coding rules to follow:

1. Use descriptive and appropriate names for all identifiers (variables, method names, class names, constants, etc.).
2. Comment every 3-7 lines of code.
3. Be neat.

Identifier Naming and Capitalization

1. Use descriptive names for all variables, function names, constants, and other identifiers.
2. Use single letter identifiers only for the counter in loops.
3. Class names start with an upper-case letter.
4. Variable names start with a lower-case letter. (Variables include parameters, local variables, and data fields. Exception: use UPPER_CASE for constants - final variables.)
5. Method names start with a lower-case letter.
6. Multi-word identifiers are internally capitalized.
7. Do not use hyphens or underscores to separate multi-word identifiers (except for constants, which have all upper-case letters).

Comments: Classes and Methods guidelines

1. Every class and methods should be preceded with a descriptive comment using the "JavaDoc" notational convention.
2. The comment should include a description of the method, the name and description of each parameter, a description of the return value, and the name and description of any exceptions thrown within the method using Javadoc keywords and formatting.
3. The comment should describe the purpose of the class.
4. Class names start with an upper-case letter.

Class, Package, and Method Naming and Capitalization

1. Classes begin with a capital letter.
2. Packages are all lower case.
3. Methods begin with a lower-case letter.
4. Multi-word identifiers are internally capitalized in methods (CamelCase).

Program Modules

1. Lines of code should be kept short, less than 80 or 100 characters wide.
2. Each public class is contained in a separate file.
3. Each file has the name of the public class contained within it.
4. Avoid the use of the default package.

Akash Kumar

Best Practices for Building RESTful APIs with Spring Boot

1. Use Descriptive and Consistent URLs (Universal Record Locator)
2. Consider API Versioning
3. Properly Use HTTP Methods
    a. Use proper HTTP response status code.
    b. Use Proper HTTP headers for serialization formats.
    c. Use nouns for resource identification.
    d. Use HTTPS in production and enable CSRF (Cross Site Request Forgery) Protection
4. Implement Proper Error Handling.
    a. Use global/custom errors handler.
    b. You should use the HandlerExceptionResolver to define a global exception.
    c. To return a correct status code for each type of exception such as BAD_REQUEST (400) for requests with invalid data.
    d. Include all relevant information in the response body.
    e. Handling all exceptions in a standardized way
    f. Log all exceptions messages and traces for debugging and monitoring purposes.
5. Implement Pagination, sorting and Filtering if required.
6. Use database migration tools like AWS database migration tools, Flyway, Ladder, Dbmate etc.
7. Use DTOs (Data Transfer Objects)
    a. Use DTO validation to handle validation logic in our code. It makes code clean.
8. Implement Validation
9. Secure Your API like - OAuth 2.0 or JWT or OpenID Connect for authentication!
    a. Use password hashing.
10. Logging, Monitoring, and availability
    a. Write clear and concise logs that are easy to read and parse!
    b. Add monitoring alerts for failures and downtimes.
    c. Expose health check points via spring-boot-actuator.
    d. Always use SLF4J (annotate at class level with @Slf4j)
11. APIs should be fully documented.
12. Test Thoroughly
13. Cache data to improve performance if possible.
14. Finally, microservices should be kept as small as possible as only one business function per service.
15. Always use the latest releases by regularly upgrading the dependencies.
16. Follow Open API 3.0 specification and check the latest updates in documentation.

Akash Kumar

   a. It automatically generates documentation in JSON/YAML and HTML format APIs.
17. Use Mapstruct if required for java bean mapping.
18. Use wireMock for unit testing if you want to simulate network calls.
19. Use Profiling in a spring-boot project.
20. Keep Controllers Simple and Clean
   a. Controllers should not execute business logic but rely on delegation.
   b. Controllers should be oriented around a use case or business capability.
   c. Controllers should deal with the HTTP layer of the application. This should not be passed down to Services.
21. Focus Services on Business Logic
   a. Constructor injection keeps code clean, does not allow us to create circular dependencies between beans.
   b. Constructor injection makes it easier to instantiate and test the class in a unit test.
22. Use Resilience4J for circuit breaker!
   a. It provides fault tolerance for our rest clients.
23. Use Backend for Frontend (BFF) as API Gateway
   a. Separation for concerns
   b. Easier to maintain and modify APIs.
   c. Better Error handling in the frontend
   d. Better security
24. Use JaCoco for Test Coverage
25. Spring Data Envers for Auditing
   a. It is used for tracking who created or changed an entity and the point in time.
26. Use Developer Tools like - spring-boot-devtools
27. Use Distributed Tracing with openTelemetry, springCloud Sleuth, and Jaeger
28. Externalize and mature your configuration management.
   a. Use a configuration server like Spring cloud config.
   b. Store all your configuration in environment variables.
29. Familiar with the concurrency model
30. Use lambda expressions and stream Api for data manipulation.

Some reference for Java coding style automation

- https://www.oracle.com/java/technologies/javase/codeconventions-contents.html
- https://google.github.io/styleguide/javaguide.html
- Introduction to Concurrency in Spring Boot | E4developer

Akash Kumar

✅ Separation of Concerns (SoC)

Divide the code into distinct sections, each addressing a separate concern.
Example: In a Spring Boot app – separate layers for controller, service, repository.

✅ 3. Fail Fast Principle

Fail quickly when an error occurs, rather than allowing it to propagate.

```
if (input == null) {
    throw new IllegalArgumentException("Input can't be null");
}
```

✅ Law of Demeter (LoD) – Principle of Least Knowledge

```
// ❌ Violates LoD
order.getCustomer().getAddress().getCity();

// ✅ Better
order.getCustomerCity();
```

✅ 5. High Cohesion, Low Coupling

Group related logic together (cohesion) and reduce dependency between components (coupling).

✅ Summary Table

Principle	Purpose	Key Benefit
DRY	Avoid repetition	Maintainable code
KISS	Keep things simple	Readability & speed
YAGNI	Avoid speculative features	Lean codebase
SOLID	OOP Design best practices	Scalability & flexibility
SoC	Divide responsibilities	Clean architecture
Fail Fast	Error detection	Debuggability
LoD	Reduce dependency	Maintainability
High Cohesion / Low Coupling	Structure code wisely	Reusability & testing ease
DYC	Document your code	
TDD	Test Driven Development	

Akash Kumar

✅ 1. Vulnerability Check in System & Code

Vulnerabilities are potential security risks in your application or infrastructure.

🔐 Security Vulnerability Check Methods:

Tool/Method	Purpose	Example Tools
Static Analysis (SAST)	Analyze source code for security flaws without running it	SonarQube, Checkmarx, Fortify
Dependency Scanning	Detect vulnerabilities in libraries/dependencies	OWASP Dependency-Check, Snyk, GitHub Dependabot
Container Image Scan	Find CVEs in Docker/OCI images	Trivy, Clair, Anchore, Docker Scout
Dynamic Analysis (DAST)	Scan running apps for exploitable issues	OWASP ZAP, Burp Suite
Infrastructure as Code (IaC) Scan	Check Terraform, Helm, YAML for misconfigurations	Checkov, Terrascan, KICS
Network Scan	Check open ports, weak services	Nmap, Nessus, OpenVAS

✅ 2. Code Smell Detection

Code smells are signs of poor design for maintainability issues.

🧹 Code Smell Check Methods:

Tool/Method	Description	Example Smells Detected
SonarQube	Detects bugs, smells, and security issues in code	Long methods, duplicate code, god classes
PMD	Static analyzer for Java, Apex, etc.	Unused variables, empty catch blocks
Checkstyle	Code style and convention checker	Naming, brace formatting
IDE Plugins	IDEs like IntelliJ and Eclipse show smells inline	Dead code, complexity, nested loops

📊 Best Practices

✅ Integrate scanning in CI/CD pipeline.
✅ Use SBOM (Software Bill of Materials) to track dependencies.
✅ Monitor & patch known CVEs.
✅ Fix code smells early to avoid tech debt.

Akash Kumar

⚙️ Example Setup: SonarQube + Trivy

1. **Code Analysis:**
   - Install SonarQube.

```
mvn sonar:sonar \
  -Dsonar.projectKey=my-app \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.login=your_token
```

   - Use Maven or Gradle plugin:

2. Image Scan (Trivy):

```
trivy image my-app:latest
```

Code Quality Metrics

Metric	Description	Good Indicator	Tools
Cyclomatic Complexity	Measures the number of independent paths in the code	Low (1–10)	SonarQube, PMD
Code Coverage	% of code exercised by automated tests	≥ 80% (depends on criticality)	JaCoCo, Cobertura
Technical Debt Ratio	Time to fix issues vs time to build the codebase	< 5% is good	SonarQube
Code Duplication	Measures identical or similar code blocks	< 5% duplication	SonarQube
Lines of Code (LOC)	Total lines of code in project or file	Smaller = better readability	SonarQube, cloc
Comment Density	Ratio of comments to code lines	~20% helpful comments	SonarQube
Maintainability Index	Composite score of readability, complexity, etc.	> 70 (Good)	Visual Studio, NDepend
Defect Density	Bugs per 1,000 lines of code	Lower is better	Bug tracking + LOC
Code Churn	Frequency of code changes over time	Moderate/low churn is ideal	Git, SonarQube
Test Pass Rate	% of tests passed	Close to 100%	JUnit, TestNG, CI/CD logs

Akash Kumar

🛠️ Tools to Measure Code Quality

Category	Tools
Static Code Analysis	SonarQube, PMD, Checkstyle, ESLint
Test Coverage	JaCoCo, Istanbul, Cobertura
CI Integration	Jenkins, GitHub Actions, GitLab CI
IDE Plugins	IntelliJ Code Metrics, VS Code extensions

🧠 Best Practices

- Set a CI pipeline gate: e.g., **"Build fails if coverage < 80%"**
- Track quality trends over time — regressions are red flags.
- Fix smells and complexity during code reviews.
- Refactor high-complexity and high-churn files first.

✅ Software Quality Check Metrics

1. Code Quality Metrics

Metric	Description	Ideal Value
Code Coverage	% of code covered by tests	≥ 80%
Cyclomatic Complexity	Number of independent paths	< 10 per method
Code Duplication	Repeated code blocks	< 5%
Code Smells	Maintainability issues in code	0 or minimal
Technical Debt Ratio	Cost to fix issues vs dev time	< 5%
Comment Density	Comments per 100 lines	15–25%

2. Testing Metrics

Metric	Description	Ideal
Test Pass Rate	% of passed test cases	≥ 95%
Defect Density	Bugs per 1K lines of code	< 1
MTTF (Mean Time to Failure)	Avg time app runs before failing	Higher is better
MTTR (Mean Time to Repair)	Time to fix a bug	Lower is better
Test Case Coverage	% of requirements tested	≥ 90%

Akash Kumar

3. Process Metrics

Metric	Description	Goal
Code Review Coverage	% of PRs/code changes reviewed	100%
Build Success Rate	CI/CD build success over time	> 98%
Lead Time	Time from code commit to production	As low as possible
Cycle Time	Time to complete a task or ticket	Shorter is better
Deployment Frequency	How often you release	Daily/Weekly (Agile)

4. Security & Compliance Metrics

Metric	**Description**	**Goal**
Vulnerability Count	Known security flaws in code	0 Critical/High
Open CVEs in Dependencies	Issues in libraries	0 Critical
Security Hotspots	Areas needing review	Close regularly

🛠️ Tools That Support These Metrics

➔ **SonarQube** – code quality, smells, complexity, duplication
➔ **JaCoCo / Istanbul** – test coverage
➔ **OWASP Dependency-Check / Snyk / Trivy** – security
➔ **Jira / Azure DevOps / GitHub Projects** – lead time, cycle time
➔ **Jenkins / GitHub Actions / GitLab CI** – build success rate

Akash Kumar